

Problem Title

Best Time to Buy and Sell Stock

Difficulty Level: Easy

Problem Description

You are given an integer array `prices` where `prices[i]` is the price of a given stock on the *i*-th day. You want to maximize your profit by choosing **one day to buy** one stock and **a later day to sell** that stock. Return the **maximum profit** you can achieve from this **single transaction**. If it's not possible to make any profit (all prices fall), return 0.

Example

Input: `prices = [7, 1, 5, 3, 6, 4]`
Output: 5

Explanation:

Buy on day 2 (price = 1)

Sell on day 5 (price = 6)

Profit = 6 - 1 = 5

Input: `prices = [7, 6, 4, 3, 1]`

Output: 0

Explanation:

Prices only decrease, so no profit is possible.

Input Format

- An integer array `prices` of length `n`.
- Each element `prices[i]` represents the stock price on day *i*.

Output Format

- Return a **single integer** representing the **maximum possible profit**.

Constraints

$1 \leq \text{prices.length} \leq 10^5$

$0 \leq \text{prices}[i] \leq 10^4$

Topic Covered

- Arrays

- Greedy / Sliding Window
- Time and Space Complexity Analysis

Approaches

1. Brute Force Approach

Idea:

Try all pairs of days (buy day < sell day) and compute profit. Track the maximum profit.

Algorithm:

```
maxProfit = 0
for i from 0 to n-1:
    for j from i+1 to n-1:
        profit = prices[j] - prices[i]
        update maxProfit
return maxProfit
```

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

When to use:

- Only for learning, small data

2. Optimized Approach (Preferred)

Idea:

Track the **minimum price** seen so far and the **maximum profit** by selling at the current price.

Algorithm:

1. Initialize minPrice = Infinity, maxProfit = 0
2. Loop through the prices
3. At each price:
 - Update minPrice
 - Compute potential profit if sold now
 - Update maxProfit

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Java Solution (Preferred Language)

```
class Solution {  
    public int maxProfit(int[] prices) {  
        int minPrice = Integer.MAX_VALUE;  
        int maxProfit = 0;  
  
        for (int price : prices) {  
            if (price < minPrice) {  
                minPrice = price;  
            } else {  
                maxProfit = Math.max(maxProfit, price - minPrice);  
            }  
        }  
  
        return maxProfit;  
    }  
}
```

Practice Link Live

- **LeetCode:**
<https://leetcode.com/problems/best-time-to-buy-and-sell-stock/> ([LeetCode](#))
- **GeeksforGeeks (similar problem):**
<https://www.geeksforgeeks.org/dsa/best-time-to-buy-and-sell-stock/> ([GeeksforGeeks](#))

Video Solution Link (YouTube)

 <https://www.youtube.com/watch?v=1pkOgXD63yU>

Learning Outcome

After solving this problem, the learner will be able to:

- Understand greedy scanning with min tracking
- Differentiate brute force vs optimized
- Write an efficient Java solution with minimal space
- Analyze time and space complexities